Importing the SDK library and build setup

Import the `samsung-health-data-api.aar` library into your app project's `libs` folder and add it into the module level `build.gradle` of your app project. Include also Gson dependency.

```
dependencies {
    implementation(fileTree(mapOf("dir" to "libs",
        "include" to "health-data-api-1.0.0.aar")))
    implementation "com.google.code.gson:gson:2.9.0"
}
```

Copy
change color theme

Apply `kotlin-parcelize` plugin to your module level `build.gradle` file.

```
plugins {
    id("kotlin-parcelize")
}
```

Getting HealthDataStore

To start utilizing Samsung Health Data SDK, get a `HealthDataStore` instance with:

- `HealthDataService.getStore()`

It helps to create the following requests:

- Checking granted data permissions and requesting data permissions.
- Reading data
- Aggregating data
- Reading data changes
- Inserting data
- Updating data
- Deleting data
- Getting DeviceManager

Permission request

Granted permission check

Accessing health data requires the user's permissions for every data type. Check that the permissions are granted using:

- `HealthDataStore.getGrantedPermissions()`

Calling this method can result in throwing `HealthDataException`. There are several types of exception classes inheriting from the mentioned parent class: `AuthorizationException`, `InvalidRequestException`, `PlatformInternalExceptio n` and `ResolvablePlatformException`. The last one of them indicates that Samsung Health is not ready to serve the specified operation. This kind of error can be resolved by the user at runtime in some cases by calling `resolve()` method.

`ResolvablePlatformException` can have one of the following error codes, which might help in applying appropriate action to the user:

- `ErrorCode.ERR_PLATFORM_NOT_INSTALLED`: Samsung Health is not installed.
- `ErrorCode.ERR_OLD_VERSION_PLATFORM`: The version of Samsung Health is old so it can't support this function of the SDK.
- `ErrorCode.ERR_PLATFORM_DISABLED`: Samsung Health has been installed but it is disabled.
- `ErrorCode.ERR_PLATFORM_NOT_INITIALIZED`: Samsung Health has been installed but the user didn't perform an initial process, such as agreeing to the Terms and Conditions.

For more details, please refer to `com.samsung.android.sdk.health.data.error` package of Samsung Health Data SDK's API Reference.

Request for permissions

If all permissions were not present, we need to request for permissions with:

- HealthDataStore.requestPermissions()

If this API call succeeds, a data permission popup will display on the app activity with a list of permissions to grant by the user. After the user allows permissions through the data permission popup, the API returns a set of granted permissions.
Below there's an example of permission popup that is shown to the user:

Below is an example code for permission checking, which covers all the topics discussed in this section:

```
suspend fun checkForPermissions(activity: Activity) {
    val permSet = mutableSetOf(
        Permission.of(DataTypes.HEART_RATE, AccessType.READ),
        Permission.of(DataTypes.STEPS, AccessType.READ),
        Permission.of(DataTypes.SLEEP_GOAL, AccessType.READ),
```

```
        Permission.of(DataTypes.SLEEP, AccessType.READ),
        Permission.of(DataTypes.BLOOD_OXYGEN, AccessType.READ),
        Permission.of(DataTypes.BLOOD_GLUCOSE, AccessType.READ),
        Permission.of(DataTypes.BLOOD_GLUCOSE, AccessType.WRITE))

    try {
        val healthDataStore =
HealthDataService.getStore(activity.applicationContext)
        val grantedPermissions =
healthDataStore.getGrantedPermissions(permSet)

        if (grantedPermissions.containsAll(permSet)) {
            // All Permissions already granted
        } else {
            // Partial or No permission, we need to request for the
permission popup
            healthDataStore.requestPermissions(permSet, activity)
        }
    } catch (error: HealthDataException) {
        if (error is ResolvablePlatformException &&
error.hasResolution) {
            error.resolve(activity)
        }
        // handle other types of HealthDataException
        error.printStackTrace()
    }
}
```

Read data

You can now retrieve data from Samsung Health using:

- `HealthDataStore.readData()`

Apart from the data type, you can also set a time filter in the request.

The example below gives a data point list (representing heart rate values) from the specific time range.

```
suspend fun readHeartRateData(startTime: LocalDateTime, endTime:
LocalDateTime) {
    val healthDataStore =
HealthDataService.getStore(applicationContext)
    val localTimeFilter = LocalTimeFilter.of(startTime, endTime)
    val readRequest = DataTypes.HEART_RATE.readDataRequestBuilder
        .setLocalTimeFilter(localTimeFilter)
        .setOrdering(Ordering.DESC)
        .build()

    val heartRateList =
        healthDataStore.readData(readRequest).dataList
    // do data processing
}
Copy

change color theme
```

The following example demonstrates how to retrieve heart rate data measured by a Galaxy Watch:

```
val localTimeFilter = LocalTimeFilter.of(startTime, endTime)
val readRequest = DataTypes.HEART_RATE.readDataRequestBuilder
    .setSourceFilter(ReadSourceFilter.fromDeviceType(DeviceGroup.WAT
CH))
    .setLocalTimeFilter(localTimeFilter)
    .setOrdering(Ordering.ASC)
    .build()
Copy

change color theme
```

To get data created in the mobile device with Samsung Health installed, set a source filter with a local device.

```
.setSourceFilter(ReadSourceFilter.fromLocalDevice())
Copy
```

You can retrieve a data's total or minimum and maximum values or the last data entry by using the function:

- HealthDataStore.aggregateData()

Total steps

To get total steps, use `DataType.StepsType.TOTAL`.
To obtain the aggregated total steps for each hour of the day, call the following function:

- `setLocalTimeFilterWithGroup()`

It takes two parameters - `LocalTimeFilter` and `LocalTimeGroup`. If the `LocalTimeFilter` is set from the beginning of today to the current time and the `LocalTimeGroup` of an hour interval is used, the result `dataList` will contain aggregated total steps for every hour of the day.

You can also sum of all elements of the `dataList` and verify that the resulting total matches the step count for the entire day as displayed in the Samsung Health Pedometer tracker.

```
private suspend fun readStepCount(startTime: LocalDateTime, endTime:
LocalDateTime) {
    val healthDataStore =
HealthDataService.getStore(applicationContext)
    val localtimeFilter = LocalTimeFilter.of(startTime, endTime)

    val readRequest = DataType.StepsType.TOTAL.requestBuilder
        .setLocalTimeFilterWithGroup(
            localtimeFilter,
            LocalTimeGroup.of(LocalTimeGroupUnit.HOURLY, 1))
        .setOrdering(Ordering.ASC)
        .build()

    val dataList =
healthDataStore.aggregateData(readRequest).dataList
    dataList.forEach {
        val hourlyStepCount = it.value
    }
    val dailyStepCount = dataList.sumOf { it.value as Long }
}
```
Copy

change color theme

Minimum and maximum heart rate values

You can retrieve minimum and maximum heart rate data from a specific time range using the following example:

```
suspend fun minHeartRateAggregateRequest(startDate: LocalDate,
endDate : LocalDate) {
    val healthDataStore =
HealthDataService.getStore(applicationContext)
    val localDateFilter = LocalDateFilter.of(startDate, endDate)
    val minHeartRateAggregate =
DataType.HeartRateType.MIN.requestBuilder
        .setLocalDateFilter(localDateFilter)
        .build()

    val dataList =
healthDataStore.aggregateData(minHeartRateAggregate).dataList
    dataList.firstOrNull().let {
        val minHR = it?.value
    }
}

suspend fun maxHeartRateAggregateRequest(startDate: LocalDate,
endDate : LocalDate) {
    val healthDataStore =
HealthDataService.getStore(applicationContext)
    val localDateFilter = LocalDateFilter.of(startDate, endDate)
    val maxHeartRateAggregate =
DataType.HeartRateType.MAX.requestBuilder
        .setLocalDateFilter(localDateFilter)
        .build()

    val dataList =
healthDataStore.aggregateData(maxHeartRateAggregate).dataList
    dataList.firstOrNull().let {
        val maxHR = it?.value
    }
}
```

Copy

change color theme

Last bedtime of sleep goal data

The aggregate operation LAST refers to the most recent values that have been saved or updated in Samsung Health.
To get the last bedtime goal, see the example below. This value is the same as that in Samsung Health > Sleep Tracker > Set target.

```
suspend fun lastBedTimeAggregateRequest(startDate: LocalDate,
endDate: LocalDate) {
    val healthDataStore =
HealthDataService.getStore(applicationContext)
    val localDateFilter = LocalDateFilter.of(startDate, endDate)
    val lastBedTimeAggregate =
DataType.SleepGoalType.LAST_BED_TIME.requestBuilder
        .setLocalDateFilter(localDateFilter)
        .build()

    val dataList =
healthDataStore.aggregateData(lastBedTimeAggregate).dataList
    dataList.lastOrNull().let {
        val lastBedTime = it?.value
    }
}
```

Read data changes

Health data in Samsung Health can be changed or deleted. If you need to get data changes for a specific data type, use:

- HealthDataStore.readChanges()

If sleep data is changed in Samsung Health, the readChanges() gives a changed data result. The result's changed item includes a changed time, a change type with ChangeType.UPSERT and the updated data point. You can retrieve deleted sleep data using ChangeType.DELETE, it includes the ID of the deleted data record.

To read sleep data change, see the following example.

```
// Read sleep data changes since the last sync time
suspend fun getChangedSleep(lastSyncTime: Instant, endTime: Instant)
{
    val healthDataStore =
HealthDataService.getStore(applicationContext)
    val changeRequest = DataTypes.SLEEP.changedDataRequestBuilder
        .setChangeTimeFilter(InstantTimeFilter.of(lastSyncTime,
endTime))
        .build()
```

```
    val changes =
healthDataStore.readChanges(changeRequest).dataList
    changes.forEach { change ->
        if (change.changeType == ChangeType.DELETE) {
            val deletedDataUid = change.deleteDataUid
        } else if (change.changeType == ChangeType.UPSERT) {
            val upsertDataPoint = change.upsertDataPoint
        }
    }
}
```

Read associated data

You can request data that is associated with a specific data.

- `HealthDataStore.readAssociatedData()`

Samsung Health measures the user's oxygen saturation and skin temperature during sleep, if the user slept with a wearable device (like Galaxy Watch). Such data is present in the Samsung Health's Sleep tracker.

After getting a specific sleep data point, reading an associated oxygen saturation data is available as in the following example.

```
private suspend fun readAssociatedOxygenSaturationData(sleepData:
HealthDataPoint) {
    val healthDataStore =
HealthDataService.getStore(applicationContext)
    val sleepSessionUid = sleepData.uid
    val readAssociateRequest =
DataTypes.SLEEP.associatedReadRequestBuilder
        .setIdFilter(IdFilter.fromDataUid(sleepSessionUid))
        .addAssociatedDataType(DataType.SleepType.Associates.BLOOD_OX
YGEN)
        .build()
    val oxygenList =
healthDataStore.readAssociatedData(readAssociateRequest).dataList
    for (oxygenData in oxygenList) {
        val oxygenDataPoints =
oxygenData.getDataPointOf(DataTypes.BLOOD_OXYGEN)
        oxygenDataPoints?.let {
```

```
            for (oxygenPoint in it) {
                val oxygen =

oxygenPoint.getValue(DataType.BloodOxygenType.OXYGEN_SATURATION)
            }
        }
    }
}
```

Insert data

To insert data into Samsung Health, use the following API and create a `HealthDataPoint` object for a specific data type you want to add:

- `HealthDataPointBuilder.builder()`

Then, you can set the data's properties. Properties of `HealthDataPoint` may vary depending on data type. Be careful to set mandatory properties.

Set the source device

If the measured health data is from a specific device like a weight scale or blood glucose meter, set the data's source device. Otherwise, the source device will be the current local device installing Samsung Health.

Find the device information in the registered devices. If no proper device, register a new device information.

Read devices registered in Samsung Health

To get a list of the devices registered in Samsung Health, you should use `DeviceManager`. The example below shows how to get information about a local device (an Android device in which Samsung Health is installed) and a list of watches that have stored their data in Samsung Health:

```
suspend fun getLocalAndWatchDevices() {
    val healthDataStore =
HealthDataService.getStore(applicationContext)
    val deviceManager = healthDataStore.getDeviceManager()

    val localDevice = deviceManager.getLocalDevice()
```

```
    Log.i(TAG, """LOCAL DEVICE:
        id: ${localDevice.id}, type: ${localDevice.deviceType},
        manufacturer: ${localDevice.manufacturer}, model:
${localDevice.model},
        name: ${localDevice.name}""".trimIndent()
    )

    val devices = deviceManager.getDevices(DeviceGroup.WATCH)
    devices.forEach { device ->
        Log.i(TAG, """DEVICE:
            id: ${device.id}, type: ${device.deviceType},
            manufacturer:${device.manufacturer}, model:
${device.model},
            name: ${device.name}""".trimIndent()
        )
    }
}
```

Copy

change color theme

Register your device into Samsung Health

As mentioned above, to set the source of inserted data, it is necessary to have your device registered in Samsung Health. To do this, you need to create a `DeviceRegistrationRequest`. See the following example:

```
fun getSeed(): String {
    return "01:A2:B3:11:C4:D5" // example MAC address used for
device seed
}

fun createDevice(): Device {
    return Device
        .accessoryBuilder()
        .addType(AccessoryType.BLOOD_GLUCOSE_METER)
        // add more types to this device, if required
        .build()
}

suspend fun deviceRegistration() {
    val deviceRegistrationRequest =
```

```
        DeviceRegistrationRequest.registerDevice(createDevice(),
getSeed())

    val healthDataStore =
HealthDataService.getStore(applicationContext)
    val deviceManager = healthDataStore.getDeviceManager()
    deviceManager.registerDevice(deviceRegistrationRequest)
}
```
Copy

change color theme

Create a HealthDataPoint object

Having our device registered, we can obtain its id at any time using provided seed. This can be done in the following way:

```
suspend fun getRegisteredDeviceId(): String {
    val healthDataStore =
HealthDataService.getStore(applicationContext)
    // get device based on MAC address used in registration
    val device =
healthDataStore.getDeviceManager().getDeviceBySeed(getSeed())
    // return id of registered device
    return device?.id ?: ""
}
```
Copy

change color theme

We can now use the obtained id to assign it while creating our new `HealthDataPoint`.

See the following example for adding new continuous glucose monitoring (CGM) data into Samsung Health. Be careful to set timestamps of each `BloodGlucose` sample correctly – series data must be enclosed between specified `startTime` and `endTime` of created `HealthDataPoint`. Otherwise, you will receive an error and you will not be able to insert the data. For example, you could consider time period of 15 minutes for the `HealthDataPoint` and add the first `BloodGlucose` entry after first 5 minutes and the second `BloodGlucose` entry after 10 minutes since given `startTime`.

Such approach is presented in the code snippet below:

```
suspend fun createBloodGlucoseDataPoint(
    startTime: Instant,
    endTime: Instant
```

```
): HealthDataPoint {
    val FIVE_MINUTES_AS_SECONDS = 5L * 60
    val TEN_MINUTES_AS_SECONDS = 2 * FIVE_MINUTES_AS_SECONDS
    val values = listOf(3.90F, 3.95F)
    val bloodGlucose = BloodGlucose.of(
        values[0],
        startTime.plusSeconds(FIVE_MINUTES_AS_SECONDS))
    val bloodGlucoseTwo = BloodGlucose.of(
        values[1],
        startTime.plusSeconds(TEN_MINUTES_AS_SECONDS))
    val seriesData = listOf(bloodGlucose, bloodGlucoseTwo)

    return HealthDataPoint.builder()
        .setStartTime(startTime)
        .setEndTime(endTime)
        .setDeviceId(getRegisteredDeviceId())
        .setClientDataId("My ID")
        // set mandatory fields
        .addFieldData(
            DataType.BloodGlucoseType.GLUCOSE_LEVEL,
            values.average().toFloat())
        .addFieldData(
            DataType.BloodGlucoseType.MEASUREMENT_TYPE,
            DataType.BloodGlucoseType.MeasurementType.WHOLE_BLOOD)
        .addFieldData(
            DataType.BloodGlucoseType.MEAL_STATUS,
            DataType.BloodGlucoseType.MealStatus.GENERAL)
        .addFieldData(
            DataType.BloodGlucoseType.SERIES_DATA,
            seriesData)
        // set other fields
        .build()
}
```

Copy

change color theme

You can now write created data into Samsung Health using:

- `HealthDataStore.insertData()`

The example code is presented below:

```
suspend fun insertBloodGlucoseData(data: HealthDataPoint) {
    val healthDataStore =
HealthDataService.getStore(applicationContext)
    val insertRequest =
DataTypes.BLOOD_GLUCOSE.insertDataRequestBuilder
        .addData(data)
        // add more if needed
        .build()

    try {
        healthDataStore.insertData(insertRequest)
    } catch (e: Exception) {
       // handle possible exceptions
       e.printStackTrace()
   }
}
```

Update data

You can also update your app's data in Samsung Health with:

- `HealthDataStore.updateData()`

After reading data, select the data's ID to update. You can use:

- `uid`: The data's ID managed in Samsung Health. It is automatically assigned by Samsung Health when a new data record is created. You can obtain it by making standard read request and accessing returned `HealthDataPoint`'s uid field.
- `clientDataId`: It indicates the ID managed in your app. It works only when the `clientDataId` has been set at the time of data insertion.

The following example shows how to update `BloodGlucoseType` data using the `clientDataId`.

```
suspend fun updateBloodGlucoseData(
    startTime: Instant,
    endTime: Instant
) {
    val healthDataStore =
HealthDataService.getStore(applicationContext)

    val FIVE_MINUTES_AS_SECONDS = 5L * 60
```

```
    val TEN_MINUTES_AS_SECONDS = 2 * FIVE_MINUTES_AS_SECONDS
    val newValues = listOf(5.4F, 5.5F)
    val bloodGlucose = BloodGlucose.of(
        newValues[0],
        startTime.plusSeconds(FIVE_MINUTES_AS_SECONDS))
    val bloodGlucoseTwo = BloodGlucose.of(
        newValues[1],
        startTime.plusSeconds(TEN_MINUTES_AS_SECONDS))
    val seriesData = listOf(bloodGlucose, bloodGlucoseTwo)

    val updatedData = HealthDataPoint.builder()
        .setStartTime(startTime)
        .setEndTime(endTime)
        .addFieldData(
            DataType.BloodGlucoseType.GLUCOSE_LEVEL,
            newValues.average().toFloat())
        .addFieldData(
            DataType.BloodGlucoseType.SERIES_DATA,
            seriesData)
        .build()

    val updateRequest =
DataTypes.BLOOD_GLUCOSE.updateDataRequestBuilder
        // updating with the app's own Data ID
        .addDataWithClientDataId(clientDataId = "My ID", data =
updatedData)
        // you may update more data entries in the request
        .build()

    try {
        healthDataStore.updateData(updateRequest)
    } catch (e: Exception) {
       // handle possible exceptions
       e.printStackTrace()
   }
}
```

Delete data

You can also delete data, inserted by you into Samsung Health, using:

- `HealthDataStore.deleteData()`

To build a delete request, it is necessary to specify the data type of the data to be deleted with it.

Remember that you can delete only the data you have inserted. Otherwise, when trying to delete the data from another app source (e.g., recorded by Samsung Health), you will get an `AuthorizationException` with an error code `2002: ERR_NO_OWNERSHIP_TO_WRITE`.

For more details regarding exceptions and error handling please refer to `com.samsung.android.sdk.health.data.error` package of the SDK's API Reference.

Below, there is an example of deleting the latest blood glucose data. To retrieve uid of the latest blood glucose data, build a read request using `ReadSourceFilter` in order get data inserted by the app. Additionally, utilize `setOrdering()` and `setLimit()` to obtain the latest data.

```
suspend fun readLatestBloodGlucoseUid(appName: String): String {
    val healthDataStore =
HealthDataService.getStore(applicationContext)
    var latestBloodGlucoseUid = ""
    val sourceFilter = ReadSourceFilter.fromApplicationId(appName)
    val readRequest = DataTypes.BLOOD_GLUCOSE.readDataRequestBuilder
        .setSourceFilter(sourceFilter)
        .setOrdering(Ordering.DESC)
        .setLimit(1)
        .build()

    try {
        val readResponse = healthDataStore.readData(readRequest)
        val responseDataList = readResponse.dataList

        if (responseDataList.isNotEmpty()) {
            latestBloodGlucoseUid = responseDataList.first().uid
        }
    } catch (e: Exception) {
        // handle possible exceptions
        e.printStackTrace()
    }
```

```
    return latestBloodGlucoseUid
}
```
Copy

change color theme

If we successfully obtained `uid`, we can use it to delete the data by making a delete request with applied `IdFilter`.

```
suspend fun deleteBloodGlucoseData(bloodGlucoseUid: String) {
    val healthDataStore =
HealthDataService.getStore(applicationContext)
    val idFilter = IdFilter.fromDataUid(bloodGlucoseUid)
    val deleteRequest =
DataTypes.BLOOD_GLUCOSE.deleteDataRequestBuilder
        .setIdFilter(idFilter)
        .build()

    try {
        healthDataStore.deleteData(deleteRequest)
    } catch (e: Exception) {
        // handle possible exceptions
        e.printStackTrace()
    }
}
```